
FONDAMENTI DI PROGRAMMAZIONE B

Tempo a disposizione: 2 ore 15 minuti

Nome Cognome Matricola

Esercizio 1 [C++] (15pt). Definire una classe template `Set<T>` che realizza il tipo di dato astratto insieme di elementi di tipo `T`. La classe deve definire:

- ▶ un costruttore senza argomenti che crea un insieme vuoto
- ▶ un metodo `void add(T value)` che aggiunge un elemento all'insieme. Se l'elemento è già presente, il metodo deve lanciare un'eccezione
- ▶ un metodo `int size() const` che ritorna la cardinalità dell'insieme
- ▶ un metodo `bool contains(const T& value) const` che ritorna `true` se l'elemento è presente nell'insieme su cui è invocato il metodo, `false` altrimenti.
- ▶ un metodo `Set<T> union(const Set<T>& other) const` che ritorna un nuovo insieme che rappresenta l'unione dell'insieme su cui è invocato il metodo con un altro insieme `other`.

Si sovraccarichi l'operatore `<<` in modo tale che stampi gli elementi dell'insieme su uno stream di output `fout` nel formato $\{e_1, e_2, \dots, e_n\}$. Non è consentito utilizzare classi della STL. Se necessario, ridefinire gli opportuni metodi, costruttori e/o operatori. Specificare opportunamente eventuali metodi e parametri costanti. Massimizzare incapsulamento e information hiding.

Esercizio 2 [Java] (15pt). Nel contesto dello sviluppo di un sistema di gestione per un portafoglio di investimenti, si implementino le seguenti classi.

- ▶ Si implementi una classe astratta che rappresenta un asset d'investimento con i seguenti attributi:
 - `String name`, che rappresenta il nome dell'asset
 - `double purchasePrice`, che rappresenta il prezzo di acquisto
 - `double quantity`, che rappresenta la quantità posseduta.

La classe deve implementare:

- un costruttore per inizializzare gli attributi
- un metodo getter per ciascun attributo
- un metodo astratto `int getRiskLevel()` che restituisce il livello di rischio dell'asset
- un metodo astratto `double getCurrentValue()` che restituisce il valore corrente dell'asset
- un metodo `double calculateProfit()` che calcola il profitto o la perdita come:

$$(\text{getCurrentValue}() - \text{purchasePrice}) * \text{quantity}$$

Si ridefinisca inoltre il metodo `equals`: due asset d'investimento sono uguali se hanno lo stesso nome.

Le classi `Stock` (azioni) e `Bond` (obbligazioni) estendono la classe `InvestmentAsset`. Si implementi solamente la classe `Stock`, di seguito descritta.

- ▶ La classe `Stock` estende `InvestmentAsset` e rappresenta un'azione e ha i seguenti attributi aggiuntivi:
 - `String sector`, che rappresenta il settore di appartenenza
 - `double currentPrice`, che rappresenta il prezzo corrente

La classe `Stock` deve:

- implementare un costruttore per inizializzare tutti gli attributi (inclusi quelli ereditati)
- implementare il metodo `double getCurrentValue()` come `currentPrice * quantity`
- implementare il metodo `int getRiskLevel()` secondo la seguente logica: 3 se il settore è "Tech", 2 se il settore è "Healthcare", 1 altrimenti.

- La classe `Portfolio` rappresenta un portafoglio d'investimenti appartenente a un proprietario e ha i seguenti attributi:
 - `String id`, che rappresenta un identificativo univoco del portafoglio
 - `String owner`, che rappresenta il nome del proprietario del portafoglio
 - `Set<InvestmentAsset> assets`, che rappresenta l'insieme degli asset d'investimento presenti nel portafoglio

La classe deve implementare:

- un costruttore che inizializza gli attributi `id` e `owner`. Quando costruito, l'insieme degli asset del portafoglio è vuoto
 - un metodo `void addAsset(InvestmentAsset asset)` che aggiunge un asset al portafoglio. Se è già presente un asset con lo stesso nome, il metodo deve lanciare un'eccezione di tipo non controllato
 - un metodo `double calculateTotalValue()` che restituisce la somma dei valori correnti (`getCurrentValue()`) di tutti gli asset presenti nel portafoglio
 - un metodo `Set<InvestmentAsset> getAssetsByRiskLevel(int level)` che restituisce l'insieme degli asset presenti nel portafoglio il cui livello di rischio (restituito da `getRiskLevel()`) è uguale al valore specificato
- (+2pt) La classe `Portfolio` deve essere iterabile (deve implementare l'interfaccia `Iterable<InvestmentAsset>`) in modo da poter iterare sugli asset del portafoglio.

N.B. Massimizzare incapsulamento e information hiding. Non è richiesta l'implementazione del metodo `hashCode` per le classi richieste. Per ciascuna classe, è possibile supporre di avere a disposizione un'implementazione del metodo `hashCode` coerente col metodo `equals` che implementerete.